0) Global architectural decisions

- **Word size**: `XLEN = 32`
- **Endianness**: `little`
- **Address size**: **32-bit address space**, but **64 MB physically implemented** (mapped).
- **Instruction size**: `32` bits
- **PC increment**:
    - `PC += ILEN/8 bytes` (byte-addressed)
- **Alignment rules**:
    - Are instructions always aligned? yes
    - Are word loads/stores aligned? yes
- **Reset behavior**:
    - initial PC value (`RESET_PC`) = 32'hFFFF_0000
    - register reset values (zeroed on reset)

1) Memory model (must be explicit)

Instruction memory vs data memory

- **Unified** (dual port)
- can do **fetch + load/store** in same cycle

Memory interface protocol

Declare the exact handshake:

- **Ready/valid handshake**:
    - `mem_ready` (always on for now)

Memory access granularity

- Byte-addressable
- Supported access sizes:

Loads

- ✅ `LB` (load byte, **sign-extend** to 32)
- ✅ `LBU` (load byte, **zero-extend** to 32)
- ✅ `LH` (load halfword, **sign-extend** to 32)
- ✅ `LHU` (load halfword, **zero-extend** to 32)
- ✅ `LW` (load word, 32-bit)

Stores

- ✅ `SB` (store byte)
- ✅ `SH` (store halfword)
- ✅ `SW` (store word)

- Load behavior:
    - `LB`: sign-extend 8 → 32
    - `LBU`: zero-extend 8 → 32
    - `LH`: sign-extend 16 → 32
    - `LHU`: zero-extend 16 → 32

- `LW`: no extension

In symbols:

- `LB`: `rd = {{24{b7}}, b[7:0]}`
- `LBU`: `rd = {24'b0, b[7:0]}`
- `LH`: `rd = {{16{h15}}, h[15:0]}`
- `LHU`: `rd = {16'b0, h[15:0]}`

- Misaligned policy:
  - trap later, undefined for now

2) Register file spec

- **Number of registers**: `32`
- **Register index width**: `REGW = log2(32)`
- **Read ports**: 2
- **Write ports**: 1
- **Is there a hardwired zero register? yes**
  - `r0 = 0 always`
- **Writeback timing**
  - write on rising edge? yes!!!!
  - read combinational? yes
    Show new value if read and write address are same in 1 cycle

If `rd_we` and `rs1_addr == rd_addr`, output `rd_wdata` on `rs1_data` (same for rs2)

3) Flags / condition codes

Model A: condition flags (classic)

- **Z (Zero)** — result == 0
- **N (Negative)** — MSB of result (sign bit)
- **C (Carry)** — unsigned carry / borrow
- **V (Overflow)** — signed overflow

- Which instructions update flags?

only explicit flag-setting instructions update flags

Flag-updating instructions:

- `ADD`, `SUB` **when encoded with "S" bit** (optional)
- `CMP` (subtract, discard result, update flags)
- `TEST` (AND, discard result, update flags)

Non-flag instructions:

- normal `ADD`, `SUB`, `AND`, `OR`, `XOR`, shifts, etc.
- loads/stores
- moves
- jumps
    -

- Branches based on which flag combinations?

Minimum branch set (enough for DOOM + OS)

- You should support these conditions:

Branch	Condition
`BEQ`	Z == 1
`BNE`	Z == 0
`BLT`	N ≠ V (signed <)
`BGE`	N == V (signed ≥)
`BLTU`	C == 0 (unsigned <)
`BGEU`	C == 1 (unsigned ≥)

4) ISA scope (what instructions exist)

A) Opcode space

- **Opcode width**: `6` bits
- **How many primary opcodes**: `2^6`
- We will use:
    - opcode + funct fields (“secondary opcodes”)

B) Required instruction categories (minimum set)

You must decide which instructions exist in each category:

Data movement

- `MOV rd, rs` (reg–reg)
- `ADDI rd, rs, imm` (load small immediate)
- `LUI rd, imm20` (upper immediate)

Integer ALU

- `ADD, SUB`
- `AND, OR, XOR`
- Shifts: `SLL, SRL, SRA`
- `NOT` omitted (use `XOR rd, rs, -1`)

Compare / set

- `CMP rs1, rs2` (sets flags)
- `TEST rs1, rs2` (sets flags)
- No `SLT/SEQ` register results (flags-based model)

Branch / jump / call

- `J imm` (unconditional)
- `JAL rd, imm`
- `JALR rd, rs1, imm`
- Conditional branches: `BEQ, BNE, BLT, BGE`
- `CALL/RET` via conventions (`JAL` / `JALR`), not separate opcodes

Load / store

- Loads: `LB, LBU, LH, LHU, LW`
- Stores: `SB, SH, SW`
- Addressing mode: **base + signed immediate offset only**

System / control

- `NOP`
- `HALT`

Explicitly excluded:

- `MUL`, `DIV`
- Floating point
- Atomics / RMW operations

5) Instruction format (bit layout)

This is *the* key thing the CU needs.

Fixed-length vs variable-length

- Fixed-length only!!!

Field positions

Use the standard “RISC-like” placement so decoding is easy:

- **[31:26]** = `opcode` (6 bits)
- **[25:21]** = `rd` (5 bits)
- **[20:16]** = `rs1` (5 bits)
- **[15:11]** = `rs2` (5 bits) *(when used)*
- The low bits become `funct / imm` depending on format

This keeps rd/rs1/rs2 in the same spots across formats.

How many formats?

1) R-type (reg–reg ALU / shifts)

Layout

- `[31:26] opcode`
- `[25:21] rd`
- `[20:16] rs1`
- `[15:11] rs2`
- `[10:6] shamt_or_subfunct` (5 bits)
- `[5:0] funct` (6 bits)

Use

- `ADD, SUB, AND, OR, XOR`
- `SLL, SRL, SRA` (use shamt or use rs2; pick one style below)

Shift style (recommended):

- Use `rs2` as the shift amount source for reg-reg shifts (`SLL rd, rs1, rs2`)
- Use `shamt` for immediate shifts (see I-type shifts below)

So for R-type shifts, `shamt_or_subfunct` can be 0 and `funct` selects the op.

---

2) I-type (reg–imm: ADDI, loads, JALR, shift-immediate)

Layout

- `[31:26] opcode`
- `[25:21] rd`
- `[20:16] rs1`
- `[15:0] imm16`

Use

- `ADDI`
- `LB/LBU/LH/LHU/LW` (base+offset)
- `JALR` (base+offset target)
- shift-immediate versions: `SLLI/SRLI/SRAI` (interpret imm[4:0] as shamt)

Immediate rules

- `imm16` is **sign-extended** for address offsets and ADDI
- For shift-immediate:
    - `shamt = imm[4:0]`
    - upper imm bits must be 0 (or used as funct for shift type)

---

3) S-type (store: base + offset)

Layout

- `[31:26] opcode`
- `[25:21] rs2` (store data)
- `[20:16] rs1` (base)
- `[15:0] imm16` (offset)

Use

- `SB/SH/SW`

Immediate rules

- `imm16` is **sign-extended**
- Effective address: `EA = rs1 + signext(imm16)`

---

4) B-type (branch: PC-relative, signed)

Layout

- `[31:26] opcode`
- `[25:21] rs1`
- `[20:16] rs2` *(optional; see note)*
- `[15:0] imm16` (PC-relative offset)

Use

- If you want **branch compares via regs**: BEQ can compare rs1 vs rs2
- But you decided **flags-based branches**. That means rs fields aren't required for BEQ/BNE (they check Z set by CMP). Still, keeping rs fields available lets you optionally support "compare-and-branch" later.

Immediate rules

- `imm16` is **sign-extended**
- Offset is in **bytes**:
    - `PC_next = PC + signext(imm16)`
- (Optional) You may define branch base as `PC+4` instead of `PC`—just pick one and document it.

Recommendation: use `PC_next = PC + 4 + signext(imm16)` (common and avoids branch-to-self footguns)

---

5) J-type (jump / JAL: PC-relative, long)

To get a useful jump range, you want **more than 16 bits**.

Layout

- `[31:26] opcode`

- `[25:21] rd` (for JAL; use r0 for plain J)
- `[20:0] imm21`

Use

- `J` is `JAL r0, imm`
- `JAL rd, imm`

Immediate rules

- `imm21` is **sign-extended**
- Offset is in **bytes**:
    - `PC_next = PC + 4 + signext(imm21)`
- Range: ±1 MiB-ish (2^{20} bytes) — enough for most programs at this scale.

---

6) U-type (upper immediate: LUI / AUIPC)

Layout

- `[31:26] opcode`
- `[25:21] rd`
- `[20:0] imm21`

Use

- `LUI rd, imm` → `rd = imm21 << 11` (example shift)
- `AUIPC rd, imm` → `rd = PC + (imm21 << 11)` (optional)

Why 21 bits?
Because opcode+rd consumes 11 bits, leaving 21.

Shift amount for U-type: choose a constant shift so you can form 32-bit constants easily.

Recommendation: `<< 11` is awkward. Better: make U-type shift by **16** like many ISAs.
To do that, you'd want imm20 ideally. Since we have imm21, you can still define:

- `rd = imm21 << 11` (fills top 21 bits) **or**
- adjust fielding to get imm20.

If you want "standard-feeling" LUI (imm20 << 12), I can tweak the field map slightly. But the rest stays identical.

Immediate details (must be explicit)

Widths

- I/S/B: **imm16**
- J/U: **imm21**

Sign-extension rules

- Branch/jump offsets: **signed**
- ADDI/load/store offsets: **signed**
- LBU/LHU: still use signed offset; zero-extension refers to loaded data, not offset

PC-relative offsets: words or bytes?

✅ **Bytes** (because memory is byte-addressed)

- Branch: `PC_next = PC + 4 + signext(imm16)`
- Jump: `PC_next = PC + 4 + signext(imm21)`

Shift amount width

- `shamt` = **5 bits** (0–31) for 32-bit data
- For shift-immediate: `shamt = imm[4:0]`
- For reg-reg shift: `shamt = rs2[4:0]` (typical)

---

6) Datapath control signals you must support (so ISA fits hardware)

Operand selection

ALU A select (`alu_a_sel`)

- `00`: `rs1`
- `01`: `PC`
- `10`: `0`
- (`11` reserved)

ALU B select (`alu_b_sel`)

- `00`: `rs2`
- `01`: `imm` (sign-extended)
- `10`: `4` (PC increment for ILEN=32)
- `11`: `imm_shifted` (for LUI/AUIPC style)

Also need:

- `imm_shift_amt` (constant, e.g. 16 or 12 depending on your LUI rule)

Writeback selection (`wb_sel`)

- `00`: `ALU_result`
- `01`: `Mem_rdata`
- `10`: `PC_plus_4` (for JAL)
- `11`: `imm_shifted` (or raw imm if you want LI as a direct path)

And:

- `reg_we` (write enable)
- `rd_sel` comes from instruction bits

PC next selection (`pc_sel`)

- `00`: `PC_plus_4`
- `01`: `PC_plus_offset` (branch/jump immediate, PC-relative)
- `10`: `reg_target` (JALR: `rs1 + imm`, with optional align mask)
- `11`: reserved

And:

- `pc_we`

Memory address selection (`mem_addr_sel`)

- `0`: `PC` (instruction fetch)
- `1`: `ALU_result` (data access effective address)

Memory command signals:

- `mem_re`
- `mem_we`
- `mem_size` (2 bits): `00=byte`, `01=half`, `10=word`
- `mem_sign` (1 bit for loads): `0=zero-extend`, `1=sign-extend`

Flags control

Since you chose flags:

- `flag_we`
- `flag_sel` (optional) if you ever want TEST to only update Z/N; otherwise just define TEST's C/V behavior in the ALU

Instruction register and handshake

- `ir_we`
- `mem_ready` input (stall if low; always 1 for now)

7) Branch/jump semantics (exact rules)

Branch condition source: flags (Z, N, C, V), not direct register compares

- Flags are set explicitly by `CMP`, `TEST`, or flag-setting ALU ops

Branch target computation:

- `PC_target = PC + 4 + signext(imm)`
- Immediate is **signed** and **byte-scaled** (no shifting)

Jump target computation:

- `J / JAL`: `PC_target = PC + 4 + signext(imm)`
- `JALR`: `PC_target = (rs1 + signext(imm)) & ~1` (optional LSB clear)

PC update timing:

- PC is updated in the **execute stage** (or equivalent control state)

Delay slots:

- **None** (branches and jumps take effect immediately)

Trap vector address

- **TRAP_VECTOR = 0xFFFF_0000** (same region as Boot ROM, convenient)

Architectural trap state (minimal CSRs)

Implement these as dedicated registers (CSR-like, even if you don't call them CSRs yet):

- **EPC** (32-bit): return PC (address of faulting instruction, unless you choose "next PC" policy)

- **CAUSE** (32-bit): reason code
- **STATUS** (32-bit): at minimum
    - `IE` (global interrupt enable)
    - `PIE` (previous IE) or a 1-bit “in_trap” to prevent nested traps (your choice)

(Optional but nice)

- **TVAL** (32-bit): trap value (bad address for misalign/access, or illegal opcode bits)

Return instruction

- **ERET** (or `RFE`):
    - `PC ← EPC`
    - restore `IE` from `PIE` (or clear `in_trap`)

What causes traps (keep it tight)

Exceptions (synchronous)

- Illegal/unknown opcode
- Misaligned instruction fetch (`PC[1:0] != 0`)
- Misaligned load/store (per your alignment rules)
- (Optional) `ECALL` / software trap

Interrupts (asynchronous)

- External interrupt line(s) (start with 1)

- (Optional later) timer interrupt

Entry semantics (exact rules)

On trap entry:

- `EPC ← PC` (or `PC+4`—pick one; I recommend **PC**)
- `CAUSE ← trap_id`
- `STATUS.PIE ← STATUS.IE`
- `STATUS.IE ← 0` (disable nested interrupts by default)
- `PC ← TRAP_VECTOR`

No delay slots.

Fixed register roles

- **r0** = `0` (hardwired zero)
- **r1** = **RA** (return address)
- **r2** = **SP** (stack pointer)
- **r3** = **FP** (frame pointer) *(optional use; can be general if you want)*

CALL / RET mechanism

- **CALL target**: `JAL r1, imm`
    - writes `PC+4` into **RA (r1)**, then jumps PC-relative

- **RET**: `JALR r0, r1, 0`
    - jumps to address in **RA**, no link (rd = r0)

Stack convention (minimal)

- Stack grows **down** (SP decremented on push / allocation)
- Function prologue/epilogue conventions can be defined later (callee-saved vs caller-saved), but CALL/RET above is the minimum required to run real code.

---

10) Performance / microarchitecture policy (affects CU)

- **ALU:** single-cycle (results registered in APU)
- **Loads/stores:** 1-cycle access (memory always-ready for now)
- **Stalling:** supported via `mem_ready` handshake (always asserted in v1)
- **Pipeline:** none in v1 (multi-cycle FSM: Fetch → Decode → Execute → WB)
- **Register writeback: next cycle** (dedicated WB stage, due to registered ALU output)